🏋️ Coding Practice and Interview Prep

Fulllion Creative Works

Contents

1. How Practice Fits In
2. Practice and Challenge Sites Compared
3. The Kinds of Technical Interviews
4. Interview Prep Sites Compared
5. LeetCode and NeetCode
6. Cracking the Coding Interview
7. A Study Plan That Works
8. Solving a Problem Out Loud
9. Behavioral Interviews and the STAR Method
10. When Are You Ready to Apply?
11. FAQ
12. Sources

---

1. How Practice Fits In

Coding challenges are like scales for a musician. They build speed and confidence with the basics: loops, strings, arrays, objects and simple algorithms. They don't replace building real projects, which is what the five recommended steps are built around.

A good balance while you're learning:

1. **Small daily challenges** (15 to 30 minutes) on one site from Section 2, in the language you're learning.
2. **Practice projects** a few times a week, from a project site such as Frontend Mentor or devChallenges, so you build whole pages and apps. For bigger portfolio pieces, see Learn Front-End Web Development's project ideas and Full-Stack's capstone ideas.
3. **Interview-style problems** (Sections 4 to 8) once you're getting close to applying for jobs.

Pick **one** challenge site to start. Jumping between sites feels productive, but it mostly means re-reading easy problems.

2. Practice and Challenge Sites Compared

The first group is for short coding exercises. The second is for building complete projects.

Coding challenges

Site	What it's for	Languages	Free or paid	Standout feature
Codewars	Short community-made challenges called "kata", ranked from beginner to expert	55+	Free. **Codewars Red** ($5 a month, or $40 a year) adds extra stats, no ads and beta features	After you solve a kata, you can compare your answer with everyone else's. Run by Andela, and built on Qualified, an assessment platform some companies use to test candidates
CodinGame	Learning through games: write a bot and send it into an arena, or solve puzzles	Many (Python, Java, C++, JavaScript and more)	Free to play	**Clash of Code**: short multiplayer races, plus bot battles, code golf (shortest code wins) and optimization contests
HackerRank	Practice problems by topic, and an Interview Preparation Kit	Many	Free for developers	Free skills certification tests (such as JavaScript, Python, SQL, React and Problem Solving) that you can share with employers
CSSBattle	Recreating a target image with as little CSS as possible	HTML and CSS only	Free, "and will always be free to play". **Plus** ($4.90 a month, or $59 a year) unlocks 1,200+ past targets and stats	A new CSS target every day, plus live battles against other players
CodingBat	Small warm-up problems with instant feedback	Java and Python	Free	Made by Nick Parlante of Stanford. Old and plain, but great for drilling fundamentals like strings, arrays, logic and recursion
Edabit	10,000+ "tiny" problems, with hints instead of full answers	C#, C++, Java, JavaScript, PHP, Python, Ruby and Swift	Free account to start	XP and levels that make daily practice feel like a game. Good for true beginners
Exercism	Learning a language through small exercises, run by a UK not-for-profit	85 language tracks, including JavaScript, Python and C#	"100% free, forever", in its own words	On tracks that have a syllabus, concept exercises unlock step by step, followed by practice exercises. You can also ask volunteer mentors to review your solutions. The track exercises are open source on GitHub

Practice projects

Site	What it's for	Languages	Free or paid	Standout feature
Frontend Mentor	120+ real-world front-end projects across five difficulty levels	HTML, CSS, JavaScript and frameworks	Free challenges, with one AI code review a month. **Pro** adds premium challenges, Figma design files and an AI review on every submission	You build from a professional design, then get feedback from the community. The best fit for a front-end portfolio

Site	What it's for	Languages	Free or paid	Standout feature
devChallenges	Real-life projects in front-end, full-stack and JavaScript paths, from Beginner to Upper-mid	HTML, CSS, JavaScript, React and more	Free to start, with a paid **Pro** plan	You submit a GitHub repository and a live demo, then review other people's code
Codementor DevProjects	Project ideas sorted by type (web, mobile, automation) and difficulty	JavaScript, Python, React, Node.js, Swift and more	Free community	You can view other people's solutions and ask mentors for help
Pygame	Making 2D games in Python, which is a fun way to practice loops, events and objects	Python	Free and open source	Start with Clear Code's The ultimate introduction to Pygame. See the note below

> **About Pygame.** If pygame.org's front page doesn't load, its documentation usually still does. The original `pygame` package was last released in September 2024 (version 2.6.1). Its former core developers now maintain a fork, **pygame-ce** (Community Edition), which releases regularly (2.5.8 in August 2026). It installs with `pip install pygame-ce`, and its docs are at pyga.me. Either way, install it into a virtual environment, as shown in the Python and Node Tool Environments guide, and check the exact package name before you install (see Safer Package Installs with npm and pip). For engines, game jams and shipping a whole game, see Learn Game Development.

> **About Clear Code's Pygame course.** It's a 3 hour 48 minute build of a small endless-runner game, and it assumes you already know basic Python functions and simple classes. Clear Code says the game itself is just a vehicle for learning every major part of Pygame. Jump to: how games work · frame rate · rectangles and collisions · player input · jumping and gravity · score · timers · animation · sprite classes · sound. The video installs the original `pygame` package. `pygame-ce` keeps the same `import pygame`, so the course code should mostly work on it too, but don't install both in one environment: pygame-ce's release notes say to run `pip uninstall pygame` first.

> **A practice site from a reel. Matty McTech**'s Facebook reel "Want to learn how to code?" shows a free, open source site where you pick a language such as JavaScript, C# or Python and work along a practice path from easy to expert. The site is **Exercism** (above): the reel walks through its language tracks, the Python track's concept map and its donation page.

3. The Kinds of Technical Interviews

"Technical interview" covers several different things. Knowing which one you're facing tells you what to practice.

Stage	What happens	How to prepare
Online assessment	A timed set of coding problems on a site like HackerRank, CodeSignal or Qualified, often before you speak to anyone	Practice timed problems on the same kind of site (Sections 2 and 4)
Recruiter or phone screen	A short call about your background and the role	Have a two-minute summary of who you are and what you've built
Live coding	You solve one or two problems while sharing your screen or in a shared editor, talking as you go	Practice out loud (Section 8), and do mock interviews (Section 4)
Take-home project	You build a small app or feature in a few days	Your practice projects from Section 2 are the best preparation
Practical or pair programming	You fix a bug or add a feature in an existing codebase with an engineer	Read other people's code, and contribute to open source
System design	You sketch how you'd build a larger system (mostly for mid-level and senior roles)	Learn how web apps fit together (Full Stack Web Development and Stacks)
Behavioral	Questions about how you've handled real situations	Prepare stories with the STAR method (Section 9)

4. Interview Prep Sites Compared

Site	What it offers	Free or paid
LeetCode	The biggest bank of interview-style problems, weekly contests, and discussion of each problem. See Section 5	Most problems and the contests are free. **Premium** adds company-tagged question lists and premium-only problems
NeetCode	A free roadmap of interview problems grouped by pattern, with a video solution for each. See Section 5	Roadmap, problem lists and videos are free. **NeetCode Pro** adds paid courses
HackerRank	The **Interview Preparation Kit**: problems grouped by topic, from arrays and hash maps to graphs and dynamic programming	Free
CodeSignal	Many companies use CodeSignal for their online assessments. **CodeSignal Learn** offers practice paths with an AI tutor called Cosmo	Learn has a free plan, with a paid upgrade
interviewing.io	Anonymous mock interviews with senior engineers from large tech companies, covering coding, system design, front end and behavioral, with written feedback	Mock interviews are paid. Its guides, question lists and nine free chapters of *Beyond Cracking the Coding Interview* are free
Tech Mock Interview	Mock interviews and coaching with engineers from tech companies (coding, system design and behavioral), plus resume reviews and salary negotiation help	Paid. Prices are on its site

Site	What it offers	Free or paid
AlgoExpert	A curated set of coding interview questions, each with a video explanation and written solutions, in an in-browser editor	Paid, usually yearly. Its price changes often with sales, so check the site
Interview Cake	Lessons that teach a way of thinking through problems, with step-by-step hints when you're stuck	A free 7-day email course and a free weekly problem. The full course is paid
Aced (formerly Exponent, which took over Pramp)	Free peer-to-peer mock interviews: you take turns interviewing another candidate	Free, with a monthly limit on sessions. Feedback and transcripts are paid
CS50: How to Prepare for Technical Interviews	A 2022 Harvard CS50 workshop led by Tommy MacWilliam, with slides and video, on resumes and interview strategy	Free. It's from CS50x 2022, so the page points to the newer course, but the advice still applies

Two good videos to start with:

- NeetCode, How I Got Good at Coding Interviews: how to study patterns instead of memorizing code (Section 5).
- Fireship, How to NOT Fail a Technical Interview: a comedy sketch of a live coding interview, with real lessons (Section 8).

5. LeetCode and NeetCode

LeetCode is where most people practice for algorithm-style interviews. It has thousands of problems, each tagged Easy, Medium or Hard, and you can solve them in most popular languages. The free tier covers almost everything you need. **Premium** adds lists of questions grouped by company, so it's worth considering only when you have a specific company's interview coming up.

PIRATE KING's How to Use LeetCode Effectively, from an engineer who prepared this way for Amazon and Microsoft, gives a five-step plan:

1. **Buy Premium,** for the company-specific question lists (step 5). He's open that he wasn't paid to say this.
2. **Start with the community's top 100 most-liked questions,** about 50 Easy and Medium ones. Heavily disliked questions tend to be unclear, full of tedious edge cases, or reliant on math tricks.
3. **Then go topic by topic,** sorting each topic's questions by **acceptance rate**, highest first, rather than by LeetCode's Easy, Medium and Hard labels, which he finds unreliable. Do at least 20 per topic, or until the pattern clicks.
4. **Practice with a plan:** leave Hard until you're ready, give yourself about 30 minutes on an Easy and an hour on a Medium before reading the solution, learn from the solutions and the **Discuss** tab (including how to make an accepted answer faster), and revisit any problem you needed help with.
5. **Before a specific interview,** practice that company's tagged questions, sorted by how often they've been asked. Three of the seven questions in his Microsoft interviews were on the list.

> **Our take on step 1:** at about $35 a month or $159 a year (as of late 2026; check LeetCode's own page), Premium is worth it mainly for step 5, when you have a real interview booked. Steps 2 to 4 work on the free tier, and NeetCode's free roadmap (below) gives you the topic-by-topic order.

NeetCode solves LeetCode's biggest problem: knowing what to do next. Its free roadmap arranges the most useful problems (the "NeetCode 150") by pattern, from arrays and hashing through to graphs and dynamic programming, in an order where each topic builds on the last. Every problem links to LeetCode and has a free video explanation.

How to use them together:

1. Open the NeetCode roadmap and start at the first topic.
2. Try each problem on LeetCode for about 20 to 30 minutes on your own.
3. If you're stuck, watch the video only until you get the key idea, then finish the code yourself.
4. Come back to the problem a few days later and solve it again from scratch. If you can't, you memorized it rather than learned it.

In How I Got Good at Coding Interviews, NeetCode explains where his lists come from and how he studied:

- **The Blind 75.** A list of 75 LeetCode problems, picked to cover the most patterns, that Yangshun Tay (author of the Tech Interview Handbook, then at Facebook) first posted on the Blind forum. NeetCode's longer lists build on it.
- **Some of those problems are very hard** to solve cold in 30 minutes. Don't feel bad about needing help.
- **Memorize the pattern, not the code.** He wrote one or two plain-English sentences per problem ("use a hash map and keep track of the index") instead of trying to remember 30 lines of code.
- **Group similar problems together,** so you see that, for example, many tree problems are the same recursive depth-first search.
- **Company interview posts:** LeetCode's Discuss section has posts where people share questions they were asked. They're user-written and can be years old, so treat them as a hint about topics, not a guaranteed list.

6. Cracking the Coding Interview

Cracking the Coding Interview by Gayle Laakmann McDowell has been the standard interview prep book for years. It explains how big tech companies interview, walks through the data structures and algorithms they test, and includes 189 practice questions with solutions.

- **Official site:** crackingthecodinginterview.com, with the table of contents, solutions and errata.
- **Latest edition:** The 6th edition (2015) is still the most recent. There's no 7th edition.
- **The 2025 sequel:** *Beyond Cracking the Coding Interview* (January 2025) by Gayle Laakmann McDowell, Mike Mroczka, Aline Lerner and Nil Mamano is the official follow-up. It's built around today's

interviews, with more on problem-solving patterns such as sliding windows and two pointers, plus getting noticed and negotiating offers. interviewing.io offers nine chapters free.

- **How to use it:** Nick White's How to use Cracking The Coding Interview Effectively. His approach:
  1. **Read the first chapter,** on how the interview process works, including how it differs between big companies.
  2. **Read the Big O chapter.** It's where time and space complexity finally made sense to him.
  3. **Use the rest as a reference.** Read the chapters on topics you're weak at, after watching a short video on the same topic. He'd hold off on bit manipulation, and skip system design for junior roles.
  4. **Use it for whiteboard practice.** Some of its questions aren't on the coding sites. Take the book somewhere with a whiteboard, ideally with a friend reading out the question.
  5. **Ask the recruiter what the interview involves,** then study the matching chapters.
  6. **Learn each topic from more than one source** (the book, a couple of videos, a blog) so one bad explanation doesn't mislead you.

You don't need to read either book cover to cover. Read the chapters on how interviews work and how to approach a problem, then use the rest as a reference while you work through problems.

> Get the book from the official site, a library or a bookshop. Free PDF copies passed around online are usually unauthorized, and download sites for them can carry malware.

7. A Study Plan That Works

Learn patterns, not answers

Most interview problems are variations on a few dozen patterns, such as two pointers, sliding windows, hash maps for fast lookups, binary search, tree and graph traversal, heaps and dynamic programming. If you understand *why* a pattern works, you can spot it in a problem you've never seen. If you memorize answers, a small twist will stop you.

After each problem, write one or two sentences in your notes: what pattern it used, and what clue in the problem pointed to it. The Computer Science Fundamentals guide covers the data structures and Big O analysis underneath these patterns.

Don't only grind problems

In Most Tech Interview Prep is GARBAGE. (From a Principal Engineer at Amazon), A Life Engineered, a principal engineer who has run over 800 interviews, argues that most prep advice is written for junior roles. His framework:

- **Interviews use proxies.** A day of interviews can't show how you'd really perform, so companies use tests that are hard to fake. Algorithm questions are a proxy for "can this person code". (That's also why he calls internships the best route into an entry-level job: they're a two-month interview.)

- **There are three levels:** coding, system design and leadership. Coding is necessary but not enough for senior roles.
- **System design questions are disguised behavioral questions.** "How would you design this feed?" is really probing whether you've built systems like it before.
- **Behavioral answers reveal your level.** The scope of your stories tells the interviewer whether you work at a junior, senior or principal level.

His suggested split of prep time:

Your situation	Coding	System design	Behavioral and leadership
New graduate or bootcamp	90%	Not expected	10%
Some experience	75%	Not expected	25%
Mid-level	40%	20%	40%
Senior	30%	30%	40%
Principal	20%	40%	40%

So for a first job, problem practice really is most of the work, which is what Sections 5 to 8 cover. For web development jobs especially, your projects often matter as much as your problem-solving. Keep building while you practice.

A sample week

Adjust this to the time you have. Consistency matters more than long sessions.

Day	Focus
Monday	Two problems from the current NeetCode topic
Tuesday	Project work (a feature, a bug fix, or a Frontend Mentor challenge)
Wednesday	Two problems, then re-solve one from last week
Thursday	Project work
Friday	One timed problem, solved out loud as if in an interview (Section 8)
Weekend	A mock interview (Aced, or a friend), plus one STAR story (Section 9). Rest is part of the plan too

Once interviews are booked, shift the balance toward timed practice and mock interviews. Before that, lean toward projects.

8. Solving a Problem Out Loud

In a live interview, *how* you work matters as much as whether you finish. Interviewers want to see how you think. A routine like this works for most problems:

1. **Repeat the problem** in your own words, and ask clarifying questions. What are the inputs? Can they be empty, negative or huge? What should happen with bad input?
2. **Work through an example** by hand, including an edge case.

3. **Describe a simple solution first,** even a slow one, and say its time and space cost (Big O).
4. **Look for a better approach.** Ask yourself which pattern from Section 7 fits.
5. **Write the code,** talking as you go. Use clear names.
6. **Test it** by walking through your example line by line, then the edge cases. Fix bugs out loud.
7. **Sum up** the final time and space cost, and mention anything you'd improve with more time.

If you get stuck, say what you're thinking. Interviewers often give hints, but only if they can follow you. Practice this routine alone first, then in mock interviews.

Fireship's How to NOT Fail a Technical Interview acts this out as a comedy sketch, with ThePrimeagen as the interviewer and FizzBuzz as the question. Under the jokes are useful habits:

- **Talk about the problem before writing code.** A flowchart or pseudocode lets you plan without worrying about syntax. The candidate only calms down once he does this.
- **"Always be chatting."** Explain each decision out loud. Saying something is almost always better than silence, and talking while you code takes practice.
- **If you freeze, ask a question** instead of staring at the screen, such as whether the function should take an input or loop over every case.
- **Prefer tools every language has.** The interviewer nudges him from a JavaScript type trick to the remainder (modulo) operator, `%`, which an interviewer from any language background will recognize.
- **Expect the requirements to change.** A follow-up ("now 7 should print something too") tests whether your code can grow. Building the output in a variable instead of hard-coding each combination handled it.
- **Be ready to give the Big O** of your solution, not "it feels fast".

9. Behavioral Interviews and the STAR Method

Almost every interview loop includes behavioral questions, such as "Tell me about a time you disagreed with a teammate" or "Describe a project that didn't go to plan." These questions ask for specific examples of how you've behaved in the past.

The **STAR method** is the standard way to structure an answer. MIT's career office and Amazon's interview guidance both recommend it:

Part	What to say	Tip
Situation	The context: where you were and what was going on	Keep it short. Only include what the listener needs
Task	What you needed to do, or the problem you faced	Make clear what *your* part was
Action	The steps *you* took, and why	This is the main part. Say "I", not "we"
Result	What happened, ideally with a number	"Load time dropped from 4 seconds to 1" beats "it went well". If it went badly, say what you learned

How to prepare:

1. Write down five to eight stories from projects, jobs, school or volunteering. Good topics include a hard bug, a conflict, a mistake, a deadline, learning something fast, and helping someone.
2. Write each one in STAR form, in bullet points rather than a script, so it doesn't sound memorized.
3. Practice saying each one in about two minutes.
4. Most stories can answer several questions. Match them up before the interview.
5. Check the company's values page, since many companies (Amazon's Leadership Principles are a well-known example) build their behavioral questions around them.

No work experience yet? Stories from personal projects, open source contributions, study groups and non-tech jobs all count.

A Life Engineered's advice from Section 7 applies here: answer behavioral questions by **telling a good story**, with a beginning, middle and end, because a story carries several points at once and is easy to remember. Ask "tell me about a time you disagreed with your team" of a junior and a senior engineer, and the size of the problem in their answer shows their level. A story about a formatting argument says something very different from one about changing how a whole service works.

10. When Are You Ready to Apply?

Earlier than you think, usually. Joshua Fluke's When are you ready to apply to developer Jobs? makes the case that waiting until you feel ready means waiting too long. Interviewing is a skill of its own, and the only way to fully practice it is to do real interviews. His main points:

- **The moment you start wondering "am I ready?", start applying.** No one will tell you you're ready. A rejection costs you only the time it took to apply, and you learn from each one.
- **Interviews are conversations, not yes-or-no quizzes.** Think of it as selling what you can do. If you lack a listed skill, show where you've used something close to it in a project.
- **You don't need to meet every requirement.** Job listings are often a wish list, and the real filtering happens once you're talking to someone.
- **Never offer to work for less** to make up for a gap. It lowers your pay, and the going rate for everyone.
- **In a whiteboard problem, communication counts.** Saying where you're stuck and asking for a hint is part of what they're testing.
- **Startups are often more flexible** about hiring newcomers. He mentions AngelList, whose job board has been called Wellfound since 2022.

> **What's changed since the video:** its salary figures for junior developers are from several years ago, so check current local data instead. The market is also tougher now: Indeed's Hiring Lab reported US software development job postings about 32% below their February 2020 level in January 2026. The second half of the video, on when to change jobs and why he'd refuse a counteroffer, belongs with Web Developer Careers: Portfolio, Getting Hired and Freelancing.

A rough checklist:

- [ ] You've built a few projects on your own, without following a tutorial, and they're live online
- [ ] Your GitHub and portfolio show those projects with clear READMEs
- [ ] You can solve most Easy problems and some Medium ones in your main language
- [ ] You can explain your own code and the choices you made
- [ ] You have a few STAR stories ready

If you can tick most of these, start applying, and keep learning while you do. Resumes, portfolios and the job search itself are covered in Web Developer Careers: Portfolio, Getting Hired and Freelancing.

11. FAQ

Do web developers really need LeetCode? It depends on the company. Large tech companies usually ask algorithm questions. Many smaller companies and agencies prefer take-home projects or practical tasks. Do enough to handle Easy and Medium problems comfortably, and spend the rest of your time on projects.

Should I pay for a premium plan? Not at first. LeetCode, NeetCode's roadmap and videos, HackerRank, Codewars and the free mock interviews on Aced cover everything you need. Consider paying only for something specific, such as company-tagged lists before an interview at that company, or a paid mock interview with an experienced engineer.

Which language should I use in interviews? The one you know best, as long as the company allows it. Python is popular because it's short to write. JavaScript is a natural choice for web developers.

Can I use AI tools to practice? Yes, carefully. Asking an AI to explain a solution you've already tried can help. Letting it solve problems for you skips the practice you're there for, and most interviews don't allow AI help. The Open Source and Developer Communities guide covers using AI tools while you learn.

I failed an interview. Now what? Everyone does, including experienced engineers. Write down the questions you remember straight away, work out the ones you missed, and add them to your practice list. Then apply again.

Is there a way into tech without algorithm interviews? Yes. Many smaller companies use practical tasks instead (see the first question). Help desk and IT support roles focus on troubleshooting and customer service, and certifications such as CompTIA A+ often help you past the first filter. IT Support Careers and Certifications covers that path.

12. Sources

Challenge and project sites

- Codewars and Codewars Red pricing
- Qualified
- CodinGame and Clash of Code
- HackerRank, Interview Preparation Kit and skills certifications

- CSSBattle and CSSBattle Plus
- CodingBat
- Edabit
- Exercism: About, language tracks and track syllabus docs
- Frontend Mentor and Frontend Mentor Pro
- devChallenges
- Codementor DevProjects
- pygame on PyPI, pygame-ce on PyPI, pygame-ce on GitHub and pygame-ce 2.5.8 release notes

Interview prep

- LeetCode
- NeetCode and NeetCode roadmap
- CodeSignal and CodeSignal Learn
- interviewing.io
- Tech Mock Interview
- AlgoExpert
- Interview Cake
- Aced (formerly Exponent and Pramp)
- CS50x 2022: How to Prepare for Technical Interviews
- Tech Interview Handbook: Blind 75 and best practice questions

Job market and job boards

- Indeed Hiring Lab: 2025 Q4 US Tech Labor Market Update
- Wellfound: AngelList Talent is now Wellfound

Books

- Cracking the Coding Interview, official site
- interviewing.io: nine free chapters of Beyond Cracking the Coding Interview

Behavioral interviews

- MIT Career Advising and Professional Development: Career toolkit, Interviewing
- Amazon: Interview prep for technical roles

Videos

- Clear Code, The ultimate introduction to Pygame
- PIRATE KING, How to Use LeetCode Effectively
- NeetCode, How I Got Good at Coding Interviews
- Nick White, How to use Cracking The Coding Interview Effectively
- A Life Engineered, Most Tech Interview Prep is GARBAGE. (From a Principal Engineer at Amazon)

- Fireship, How to NOT Fail a Technical Interview
- Joshua Fluke, When are you ready to apply to developer Jobs?
- Matty McTech, "Want to learn how to code?" (Facebook), a short reel on Exercism

---

Latest version: github.com/ImmuneMoon/Coding-Practice-and-Interview-Prep

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.